

Problem Set 7

BM2043 - Algorithms and Data Structure Lab

October 25, 2024

Recursion and Backtracking

Problem 1: Generate Parentheses

Problem Statement:

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Examples: Example 1:

- **Input:** $n = 3$
- **Output:** ["((()))", "(()())", "(())()", "()(())", "()()()"]

Example 2:

- **Input:** $n = 1$
- **Output:** ["()"]

Constraints:

- $1 \leq n \leq 8$

Class Definition:

```
class Solution {
public:
    vector<string> generateParenthesis(int n) {
        // Implement your solution here (using recursion or backtracking)
    }
};
```

Problem 2: Subsets

Problem Statement:

Given an integer array `nums` of unique elements, return all possible subsets (the power set).
The solution set must not contain duplicate subsets. Return the solution in any order.

Examples: Example 1:

- **Input:** `nums = [1, 2, 3]`
- **Output:** `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]`

Example 2:

- **Input:** `nums = [0]`
- **Output:** `[[], [0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are unique.

Class Definition:

```
class Solution {
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        // Implement your solution here (using backtracking or recursion)
    }
};
```

Problem 3: Number of Subsequences That Satisfy the Given Sum Condition

Problem Statement:

You are given an array of integers `nums` and an integer `target`.

Return the number of non-empty subsequences of `nums` such that the sum of the minimum and maximum element in the subsequence is less than or equal to `target`. Since the answer may be too large, return it modulo $10^9 + 7$.

Examples: Example 1:

- **Input:** `nums = [3, 5, 6, 7], target = 9`
- **Output:** 4
- **Explanation:**
 - There are 4 subsequences that satisfy the condition:
 - `[3] → (min + max) = 3 + 3 = 6 ≤ 9`
 - `[3, 5] → (min + max) = 3 + 5 = 8 ≤ 9`
 - `[3, 5, 6] → (min + max) = 3 + 6 = 9 ≤ 9`
 - `[3, 6] → (min + max) = 3 + 6 = 9 ≤ 9`

Example 2:

- **Input:** `nums = [3, 3, 6, 8], target = 10`

- **Output:** 6
- **Explanation:**
 - There are 6 subsequences that satisfy the condition:
 - $[3], [3], [3, 3], [3, 6], [3, 6], [3, 3, 6]$

Example 3:

- **Input:** $\text{nums} = [2, 3, 3, 4, 6, 7]$, $\text{target} = 12$
- **Output:** 61
- **Explanation:**
 - There are 63 non-empty subsequences in total, and two of them do not satisfy the condition: $[6, 7]$ and $[7]$. Hence, the number of valid subsequences is $63 - 2 = 61$.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^6$
- $1 \leq \text{target} \leq 10^6$

Class Definition:

```
class Solution {
public:
    int numSubseq(vector<int>& nums, int target) {
        // Implement your solution here
    }
};
```

Problem 4: Combination Sum

Problem Statement:

You are given a collection of integers candidates and a target number target. Your task is to find all unique combinations in candidates where the numbers sum to the target. Each number in candidates may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Examples: Example 1:

- **Input:** $\text{candidates} = [10, 1, 2, 7, 6, 1, 5]$, $\text{target} = 8$
- **Output:**
 - $[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]$

Example 2:

- **Input:** $\text{candidates} = [2, 5, 2, 1, 2]$, $\text{target} = 5$
- **Output:**
 - $[[1, 2, 2], [5]]$

Constraints:

- $1 \leq \text{candidates.length} \leq 100$
- $1 \leq \text{candidates}[i] \leq 50$
- $1 \leq \text{target} \leq 30$

Class Definition:

```
class Solution {
public:
    vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
        // Implement your solution here
    }
};
```

Problem 5: Letter Combinations of a Phone Number**Problem Statement:**

Given a string containing digits from ****2-9**** inclusive, return all possible letter combinations that the number could represent. Return the answer in ****any order****.

A mapping of digits to letters (similar to the telephone buttons) is as follows:

- 2 - "abc"
- 3 - "def"
- 4 - "ghi"
- 5 - "jkl"
- 6 - "mno"
- 7 - "pqrs"
- 8 - "tuv"
- 9 - "wxyz"

Note: The digit '1' does not map to any letters.

Examples: Example 1:

- **Input:** digits = "23"
- **Output:** ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

- **Input:** digits = ""
- **Output:** []

Example 3:

- **Input:** digits = "2"
- **Output:** ["a","b","c"]

Constraints:

- $0 \leq \text{digits.length} \leq 4$
- $\text{digits}[i]$ is a digit in the range $['2', '9']$.

Approach:

- Use a backtracking approach to generate all possible combinations.
- If the input is an empty string, return an empty list.
- Maintain a mapping of digits to their respective letters and use recursion to explore all combinations.

Class Definition:

```
class Solution {
public:
    vector<string> letterCombinations(string digits) {
        // Implement your solution here
    }
};
```

Problem 6: Word Search**Problem Statement:**

Given an $m \times n$ grid of characters 'board' and a string 'word', return **true** if the word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Examples: Example 1:

- **Input:**

A	B	C	E
S	F	C	S
A	D	E	E

`board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"`

- **Output:** true

Example 2:

- **Input:**

A	B	C	E
S	F	C	S
A	D	E	E

board = [["A", "B", "C", "E"], ["S", "F", "C", "S"], ["A", "D", "E", "E"]], word = "SEE"

- **Output:** true

Example 3:

- **Input:**

A	B	C	E
S	F	C	S
A	D	E	E

board = [["A", "B", "C", "E"], ["S", "F", "C", "S"], ["A", "D", "E", "E"]], word = "ABCB"

- **Output:** false

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].length$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- board and word consist of only lowercase and uppercase English letters.

Class Definition:

```
class Solution {
public:
    bool exist(vector<vector<char>>& board, string word) {
        // Implement your solution here
    }
};
```

Problem 7: N-Queens

Problem Statement:

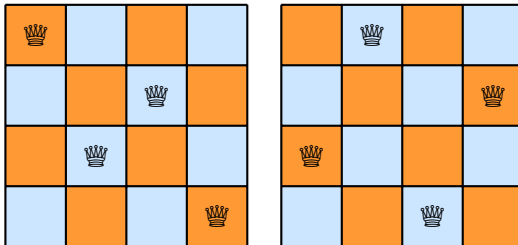
The n-queens puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other.

Given an integer n , return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Examples: Example 1:

- **Input:**



$n = 4$

- **Output:**

```
[[".Q..", "...Q", "Q...", "..Q."],  
 ["..Q.", "Q...", "...Q", ".Q.."]]
```

- **Explanation:** There exist two distinct solutions to the 4-queens puzzle as shown above.

Example 2:

- **Input:**

$n = 1$

- **Output:**

```
[["Q"]]
```

Constraints:

- $1 \leq n \leq 9$

Class Definition:

```
class Solution {  
public:  
    vector<vector<string>> solveNQueens(int n) {  
        // Implement your solution here  
    }  
};
```

Problem 8: Rat in a Maze Problem

Problem Statement:

Consider a rat placed at $(0, 0)$ in a square matrix mat of order $n \times n$. It has to reach the destination at $(n - 1, n - 1)$. Find all possible paths that the rat can take to reach from source to destination. The directions in which the rat can move are 'U' (up), 'D' (down), 'L' (left), 'R' (right).

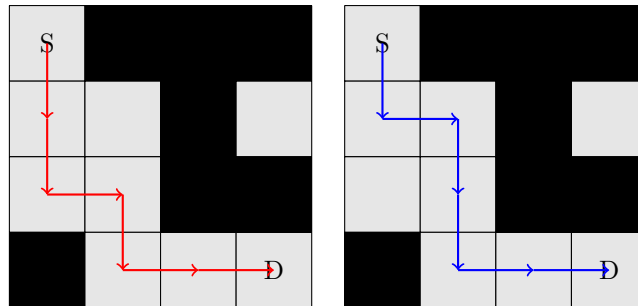
- Value 0 at a cell in the matrix represents that it is blocked and the rat cannot move to it, while value 1 at a cell in the matrix represents that the rat can travel through it. - Note: In a path, no cell can be visited more than once. If the source cell is 0, the rat cannot move to any other cell. In case of no path, return an empty list. The driver will output "-1" automatically.

Examples: Example 1:

- **Input:**

```
mat[][] = [[1, 0, 0, 0],
            [1, 1, 0, 1],
            [1, 1, 0, 0],
            [0, 1, 1, 1]]
```

- **Output:** 'DDRDRR DRDDRR'



Explanation: The rat can reach the destination at $(3, 3)$ from $(0, 0)$ by two paths - 'DRDDRR' and 'DDRDRR'. When printed in sorted order, we get 'DDRDRR DRDDRR'.

Example 2:

- **Input:**

```
mat[][] = [[1, 0],
            [1, 0]]
```

- **Output:** '-1'

Explanation: No path exists and the destination cell is blocked.

Constraints:

- $2 \leq n \leq 5$
- $0 \leq \text{mat}[i][j] \leq 1$

Class Definition:

```
class Solution {  
public:  
    vector<string> findPath(vector<vector<int>> &mat) {  
        // Your code goes here  
    }  
};
```